

OPERATOR OVERLOADING

Dr. S. Vinila Jinny

C++ Operator Overloading

- **Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined data type.**
- **Operator overloading is used to overload or redefines most of the operators available in C++.**

C++ Operator Overloading

- It is used to perform the operation on the user-defined data type.
- For example, C++ provides the ability to add the variables of the user-defined data type that is applied to the built-in data types.

Operator that cannot be overloaded are as follows:

- **Scope operator (::)**
- **Sizeof**
- **member selector (.)**
- **member pointer selector (*)**
- **ternary operator (?:)**

Syntax

```
return_type class_name :: operator op(argument_list)
{
    // body of the function.
}
```

Rules for Operator Overloading

- Existing operators can only be overloaded, but the new operators cannot be overloaded.
- The overloaded operator contains at least one operand of the user-defined data type.
- We cannot use friend function to overload certain operators. However, the member function can be used to overload those operators.

Rules for Operator Overloading

-
- **When unary operators are overloaded through a member function take no explicit arguments, but, if they are overloaded by a friend function, takes one argument.**
- **When binary operators are overloaded through a member function takes one explicit argument, and if they are overloaded through a friend function takes two explicit arguments.**

Example

```
#include <iostream>
using namespace std;
class Test
{
    private:
        int num;
    public:
        Test(): num(8){}
        //Test(){num=8;}
        void operator ++()      {    num
= num+2;    }
        void operator ++(int){
num=num+2;}
        void Print() {
            cout<<"The Count is: "<<num;
        }
};
```

```
int main()
{
    Test tt;
    ++tt; // calling of a function //void
operator ++()
    tt.Print();
    tt++;
    tt.print();
    return 0;
}
```


Example

```
#include <iostream>
using namespace std;
class A
{
    int x;
public:
    A(){}
    A(int i)
    {
        x=i;
    }
    void operator+(A); // friend void
operator+(A,A);
    void display();
};
```

```
void A :: operator+(A a)
{

    int m = x+a.x;
    cout<<"The result of the addition of
two objects is : "<<m;
}
int main()
{
    A a1(5);
    A a2(4);
    a1+a2; // a1.operator+(a2);
    return 0;
}
```

Overloading I/O operators:

- I/O operator are
 - stream extraction operator >>
 - stream insertion operator <<
- The stream insertion and stream extraction operators also can be overloaded to perform input and output for user-defined types like object.
- In overloading operator <<, the left operand is the 'cout' object and the right operand is the class object. The 'cout' is actually an object of type ostream.
- In overloading operator >>, the left operand is the cin object and the right operand is the class object cin which is an object of type istream.

Overloading I/O Operator

- The general syntax for overloading I/O operator is as follows:

Return-type operator sign (parameter list)

{

Body of function

}

- Return-type:
- The return type for overloaded stream extraction operator is ostream& and for overloaded stream insertion operator is istream&.
- There are two parameters for overloaded I/O operators. The first is of ostream type for stream extraction overloading and istream type for stream insertion overloading, and the second is of that class whose object is used.

Example

```
#include <iostream>
using namespace std;
class student
{
    private:
        int roll_no;
        float marks;
    public:
        student(int r, float m)
        {
            roll_no=r;
            marks=m;
        }
}
```

```
friend ostream& operator<<(ostream &out, student &st)
{
    out<<"roll_no: "<<st.roll_no<<endl;
    out<<"marks: "<<st.marks<<endl;
    return out;
}

};

int main()
{
    student s1(1,490), s2(2, 600);
    cout<<"Record of Studen1 is:\n";
    cout<<s1;
    cout<<"Record of Studen2 is:\n";
    cout<<s2;
}
```

Example

```
#include <iostream>
using namespace std;
class employee
{
private:
    int emp_no;
    char name[10], address[40];
public:
friend istream& operator>>(istream &in, employee
&emp)
{
    cout<<"Enter employee no: ";
    in>>emp.emp_no;
    cout<<"Enter employee Name: ";
    in>>emp.name;
    cout<<"Enter employee address: ";
    in>>emp.address;
    return in;
}
```

```
friend ostream& operator<<(ostream &out, employee
&emp)
{
    out<<"Emp no: "<<emp.emp_no<<endl<<"Name:
"<<emp.name<<endl<<"Address:
"<<emp.address<<endl;
    return out;
}
};
int main()
{
    employee e1,e2;
    cout<<"enter record for employee 1:\n";
    cin>>e1;
    cout<<"enter record for employee 2:\n";
    cin>>e2;
    cout<<"record of employee 1:\n";
    cout<<e1;
    cout<<"record of employee 2:\n";
    cout<<e2;
}
```

Overloading Comparison Operator

```
#include <iostream>
using namespace std;
class distance
{
    private:
        int feet;
        int inches;
    public:
        distance(int f, int i)
        {
            feet = f;
            inches=i;
        }
        void display()
        {
            cout<<"Feet: "<<feet<<endl<<"inches:"<<inches<<endl;
        }
        int operator< (const distance& d)
        {
            if((feet==d.feet) && (inches < d.inches))
                return 1;
            else
                return 0;
        }
};
```

```
int main()
{
    distance d1(11, 10), d2(5, 11);
    cout<<"Values of d1"<<endl;
    d1.display();
    cout<<"Values of d2"<<endl;
    d2.display();
    if(d1<d2)
        cout<<endl<<"d1 is less than d2"<<endl;
    else
        cout<<endl<<"d2 is less than d1"<<endl;
}
```

Overloading Subscript Operator:

```
#include <iostream>
#include<conio.h>
#include<string.h>
using namespace std;
const int SIZE = 10;
class saferarray
{
private:
    int arr[SIZE];
public:
    saferarray()
    {
        for(int i=0; i<SIZE; i++)
        {
            arr[i]=i;
        }
    }
}
```

```
int &operator[] (int i)
{
    if(i>SIZE)
    {
        cout<<"index out of bound"<<endl;
        return arr[0];
    }
    return arr[i];
};

int main ()
{
    saferarray a;
    cout<<"Value of A[2] : "<<a[2]<<endl;
    cout<<"Value of A[5] : "<<a[5]<<endl;
    cout<<"Value of A[12] : "<<a[12]<<endl;
}
```

Friend function cannot be used for overloading these operators

- =
- ()
- []
- ->